

# When Serverless Is the Wrong Choice

4 real limits you need to know before you build

Dr. Joanne Skiles · @chaotictoejam

01

ABOUT THIS SERIES

# 4 videos. 1 foundation.

01 When serverless is the wrong choice

← *You are here*

02 Stateless does not mean simple

03 Why serverless is about failure domains

04 Build a minimal serverless API

*hands-on build*

# What serverless actually is

## The model

**Write functions.**

Deploy to a cloud provider.

They run on demand.

You don't manage servers.

You pay per execution.

## Common platforms

**AWS Lambda**

Most popular, what we focus on

**Google Cloud Functions**

Same model, GCP flavour

**Azure Functions**

Microsoft's equivalent

FOUR REAL LIMITS

# Where serverless breaks down



01

## Cold starts

Latency spikes you can't predict



02

## Long-running processes

15-minute hard ceiling



03

## Cost at high volume

The economics flip at scale



04

## Stateful workloads

Some things need persistent memory

LIMIT 01

# Cold Starts

The latency spike everyone talks about

## WARM REQUEST

~5–15ms

execute

## COLD REQUEST

boot

init

execute

100ms – 3s



Rule of thumb: if you need consistent sub-50ms latency, serverless will frustrate you.

## WHERE THIS HURTS

- ✗ Real-time apps (chat, games, live dashboards)
- ✗ Mobile APIs where users expect instant response
- ✗ Workloads with strict p99 latency SLAs



# Long-Running Processes

Lambda has a 15-minute hard ceiling. Most work is fine. Some isn't.



Most API requests, cron jobs, event processors (ms to seconds)



15 min wall



## Video transcoding

Encoding a 2-hour film > 15 min



## Large data processing

Crunching a 10 GB CSV



## Web scraping at scale

Crawling hundreds of pages

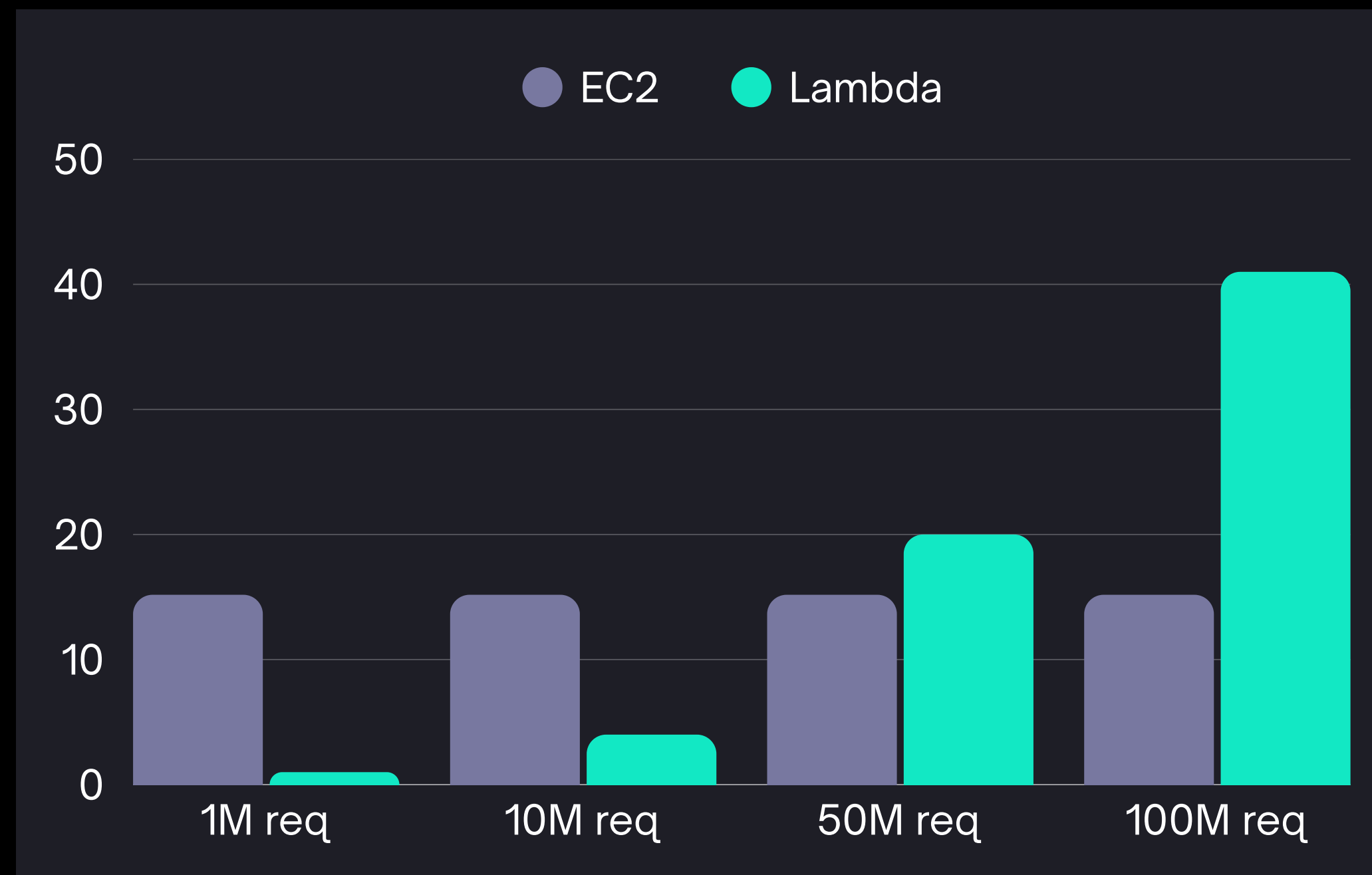


## ML model training

GPU jobs run for hours

# Cost at High Volume

Serverless is cheap at low scale. The economics flip at very high scale.



## THE MATH

128 MB

100ms per request

100M requests / month

Lambda → ~\$40.83

EC2 t3.small (on demand) →  
~\$15.18

# Stateful Workloads

When your function finishes, its memory is gone. That matters.

## EACH INVOCATION IS ISOLATED

Invocation A

no memory

Invocation A

no memory

Invocation A

no memory

## WORKLOADS THAT SUFFER

- ✗ **WebSockets**  
Persistent connections need a persistent process
- ✗ **In-memory cache**  
Each cold start rebuilds the lookup table
- ✗ **Streaming pipelines**  
Can't hold a rolling buffer between calls



THE GREEN ZONE

# When serverless IS the right choice



## **Bursty / unpredictable traffic**

Scale to zero and back up automatically. Pay for nothing at idle.



## **Event-driven processing**

File uploads, DB changes, messages in a queue



## **Scheduled jobs**

Cron-style tasks. Run once an hour, pay for one invocation.



## **Glue code & automations**

Stitching services together. Lambda is great at orchestration.



## **Side projects & prototypes**

Zero ops overhead. Ship fast, iterate, keep the free tier.

# Quick gut-check for your workload

**1** Does it need sub-50ms p99 latency consistently?

If yes → Consider containers or VMs.

**2** Does it run longer than 15 minutes?

If yes → Use ECS, Fargate, or EC2.

**3** Do you expect 100M+ requests/month on a tight budget?

If yes → Model the cost first.

**4** Does it need a persistent in-memory state?

If yes → Serverless will fight you.

## TAKEAWAYS

# What to remember from this video

- Serverless is powerful, but it's not magic
- Cold starts make tail latency unpredictable
- The 15-minute ceiling rules out long-running workloads
- At high volume the economics flip
- Stateful needs (WebSockets, in-memory caches, streaming buffers) fight the serverless model
- Understanding limits makes you a better engineer